

Solution Requirements

Date	25 September 2026
Team ID	SWTID-2026-6721
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

The functional requirements describe what the **AI FAQ Assistant API** should do, while the non-functional requirements describe how the system should perform, remain secure, and maintain reliable operation.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	<i>Users can register, log in, and access their profile. Passwords are securely hashed using bcrypt and authentication is handled using JWT tokens.</i>	High

2	Authorization levels	<i>Protected API operations require a valid JWT token. Users can create, update, and delete their own FAQ records.</i>	<i>High</i>
---	----------------------	--	-------------

3	External interfaces	<i>The system integrates with Google Gemini AI for generating answers and FAQ question-answer pairs. MongoDB is used as the database through Mongoose.</i>	<i>High</i>
4	Transactions processing	<i>The API processes requests for user registration, login, FAQ creation, updating, deletion, retrieval, and AI-generated responses.</i>	<i>High</i>
5	Reporting	<i>The system provides JSON responses containing user, FAQ, search, and AI-generated information through the API endpoints.</i>	<i>Medium</i>
6	Business rules, etc.	<i>FAQs must contain a question, answer, and valid category. Only authenticated users can perform protected operations, and FAQ ownership is verified for modifications.</i>	<i>High</i>

7	Compliance to laws or regulations	<i>User credentials and authentication information must be handled securely. API responses should not expose passwords or sensitive authentication data.</i>	<i>High</i>
8	<i>Other</i>	<i>The system provides FAQ keyword search and centralized error handling for validation, database, authentication, and API errors.</i>	<i>Medium</i>

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	<i>API requests should be processed efficiently and return responses without unnecessary delay.</i>	<i>Normal API requests should return within a few seconds under normal usage.</i>
2	Scalability	<i>The modular Express.js and MongoDB architecture should allow the system to handle increasing FAQ records and users.</i>	<i>Database and API structure should support future expansion.</i>
3	Security & Data Privacy	<i>Passwords must be hashed using bcrypt, protected routes must use JWT authentication, and API keys must be stored in environment variables.</i>	<i>No passwords or API keys should be stored directly in source code.</i>
4	Reliability & Availability	<i>The API should provide consistent responses and handle errors through centralized error-handling</i>	<i>Server errors should return structured JSON error responses.</i>

		<i>middleware.</i>	
5	Usability & Accessibility	<i>API endpoints should be clearly structured and easy to test using Postman or Thunder Client.</i>	<i>Requests and responses should follow a consistent JSON format.</i>
6	<i>Maintainability & Portability</i>	<i>The project should use a modular MVC structure with separate routes, controllers, models, middleware, and services.</i>	<i>Individual modules should be easy to modify and maintain.</i>